

◆ CS EDUCATION ◆

🔍

해시 테이블

26.1.30
박현정

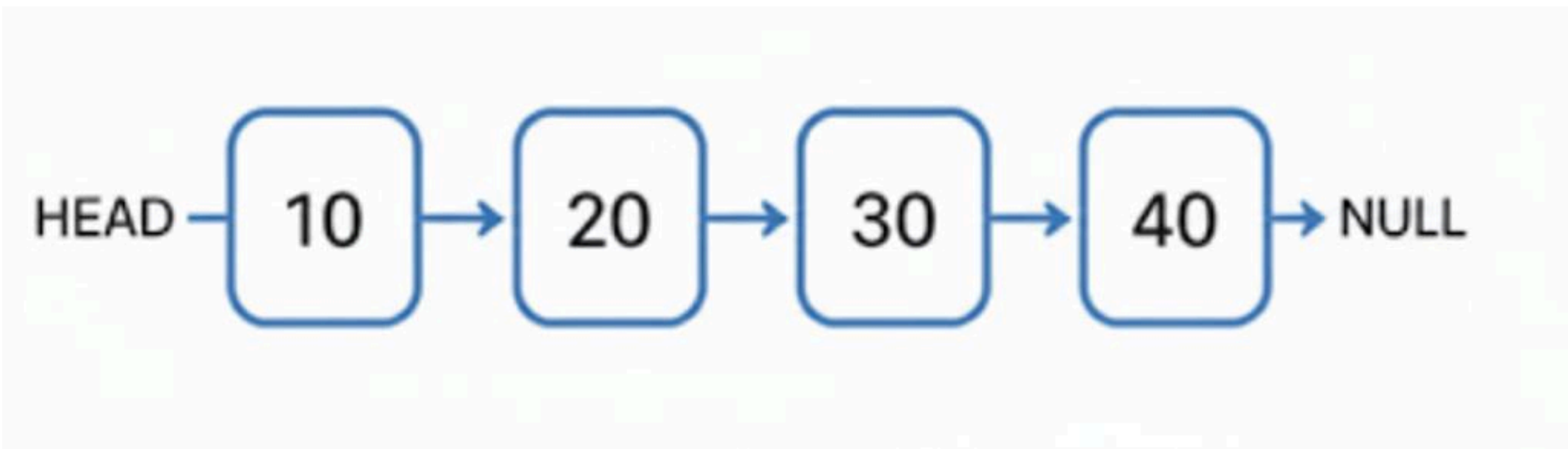
교육 목표

- 해시 테이블의 개념과 구조, 동작 원리 이해
- 해시 함수
 - 역할과 분류
 - 좋은 해시 함수의 조건
- 해시 충돌
 - 원인
 - 해결 방법

0. 선형 자료구조와 비선형 자료구조

선형 자료구조

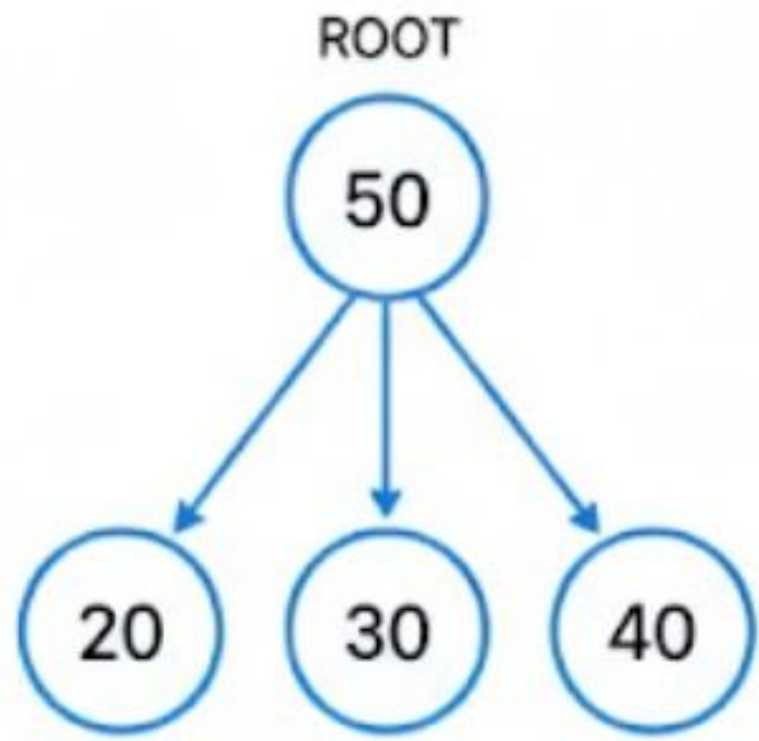
요소가 일렬로 나열되어 있는 자료구조



(ex) 배열, 연결 리스트, 스택

비선형 자료구조

요소가 일렬로 나열되어 있지 않고 자료
순서나 관계가 복잡한 구조



(ex) 트리, 그래프

0. 선형 자료구조와 비선형 자료구조

💡 공통적인 고민과 한계:

“내가 원하는 데이터를 얼마나 빨리 찾을 수 있는가?”

배열: 정렬되어 있지 않다면 처음부터 다 뒤져봐야 할 수 있음($O(n)$)

트리: 층을 타고 내려가야 함($O(\log N)$)

0. 선형 자료구조와 비선형 자료구조

💡 이에 등장한 개념

맵(Map)

- 무엇이 어디에 있는지를 키와 값의 쌍으로 저장하는 추상적인 개념
- 그리고 이 맵을 '가장 압도적인 속도'로 구현해낸 자료구조가 **'해시 테이블'**

1. 해시 테이블이란?

키(key)와 값(value)의 대응으로 이루어진 표(테이블)와 같은 형태의 자료구조

- 키 : 해시 테이블에 대한 입력
- 값 : 키를 통해 얻고자 하는 데이터

비유하자면 마치 "이름(Key)"을 알면 바로 그 사람의 "전화번호(Value)"를 찾을 수 있는 전화번호부 같은 테이블

1. 해시 테이블이란?

- 검색하고자 하는 키를 입력하면 바로 데이터가 나옴

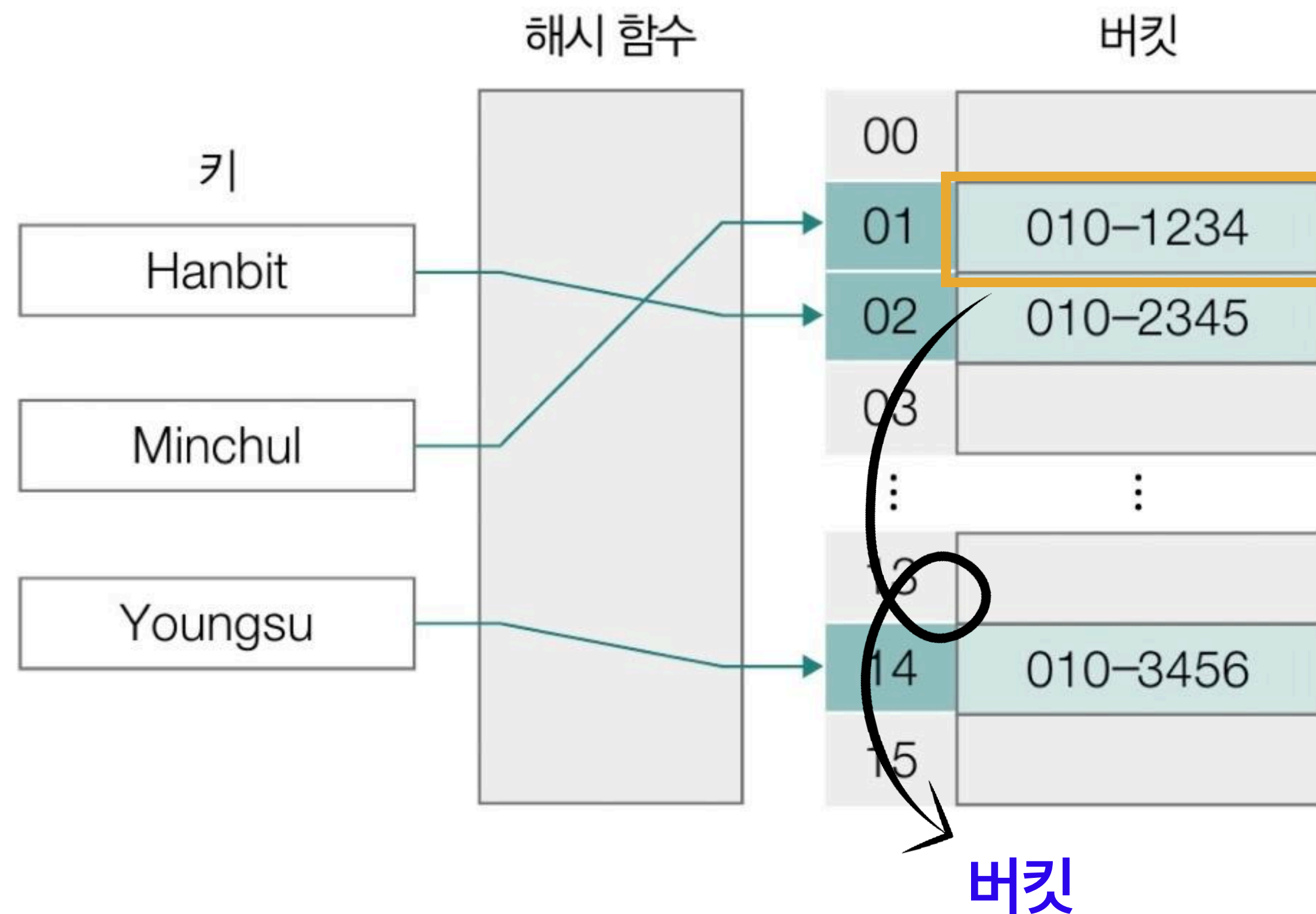
→ 매우 직관적이라는 특징

- 실제로 어디에 사용될까?

파이썬의 딕셔너리, 자바의 해시맵, 자바스크립트의 Object, 데이터베이스의 인덱싱

→ 현대 소프트웨어의 필수 기술

2. 해시 테이블의 구조와 동작



구조

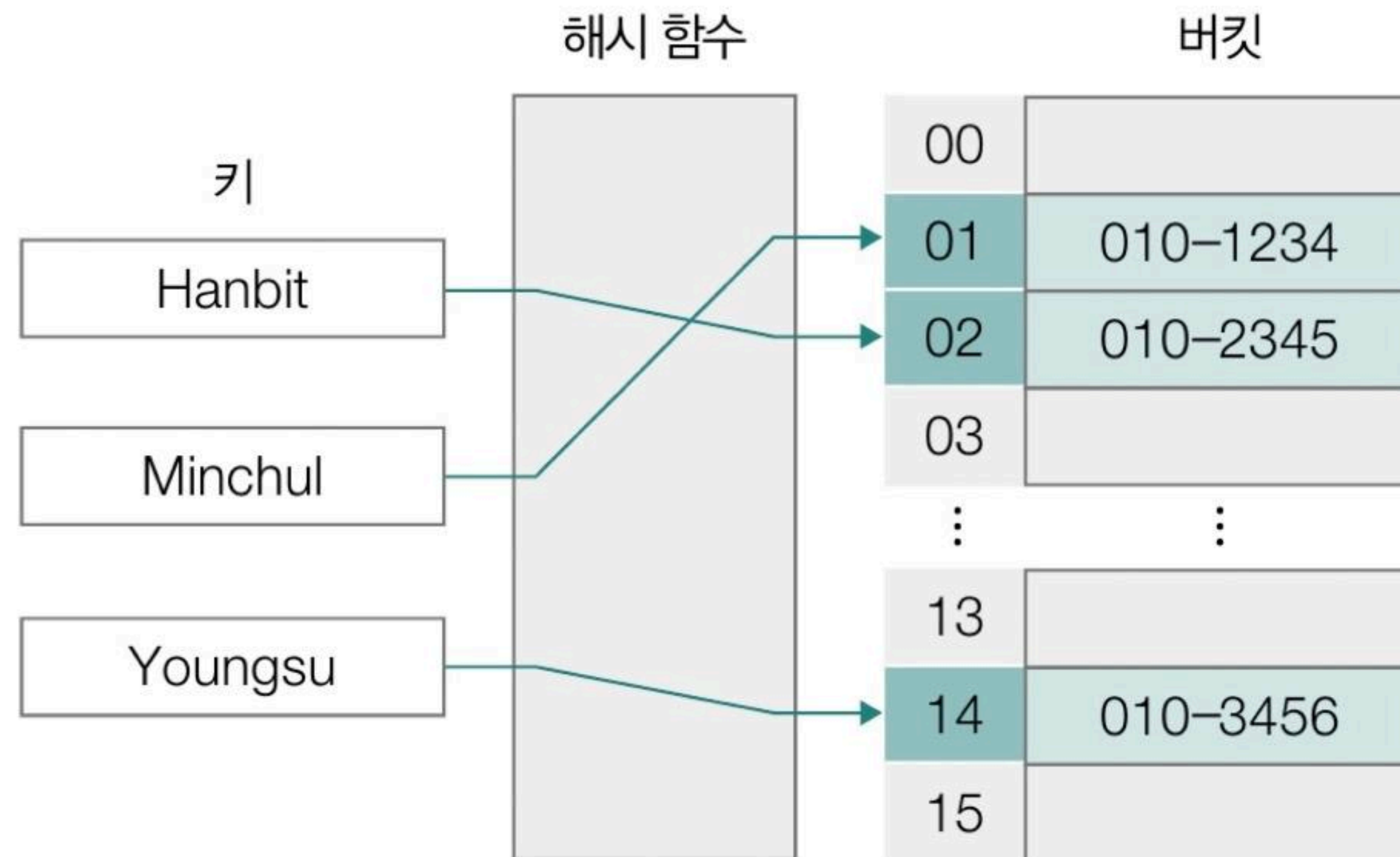
1. 해시 함수(hash function)

- 키를 입력받아 정수(인덱스)를 계산하는 함수

2. 버킷 배열(bucket array)

- 해시 함수가 계산한 인덱스에 따라 데이터가 저장되는 배열
- 배열의 각 칸 하나를 "버킷"이라고 부름

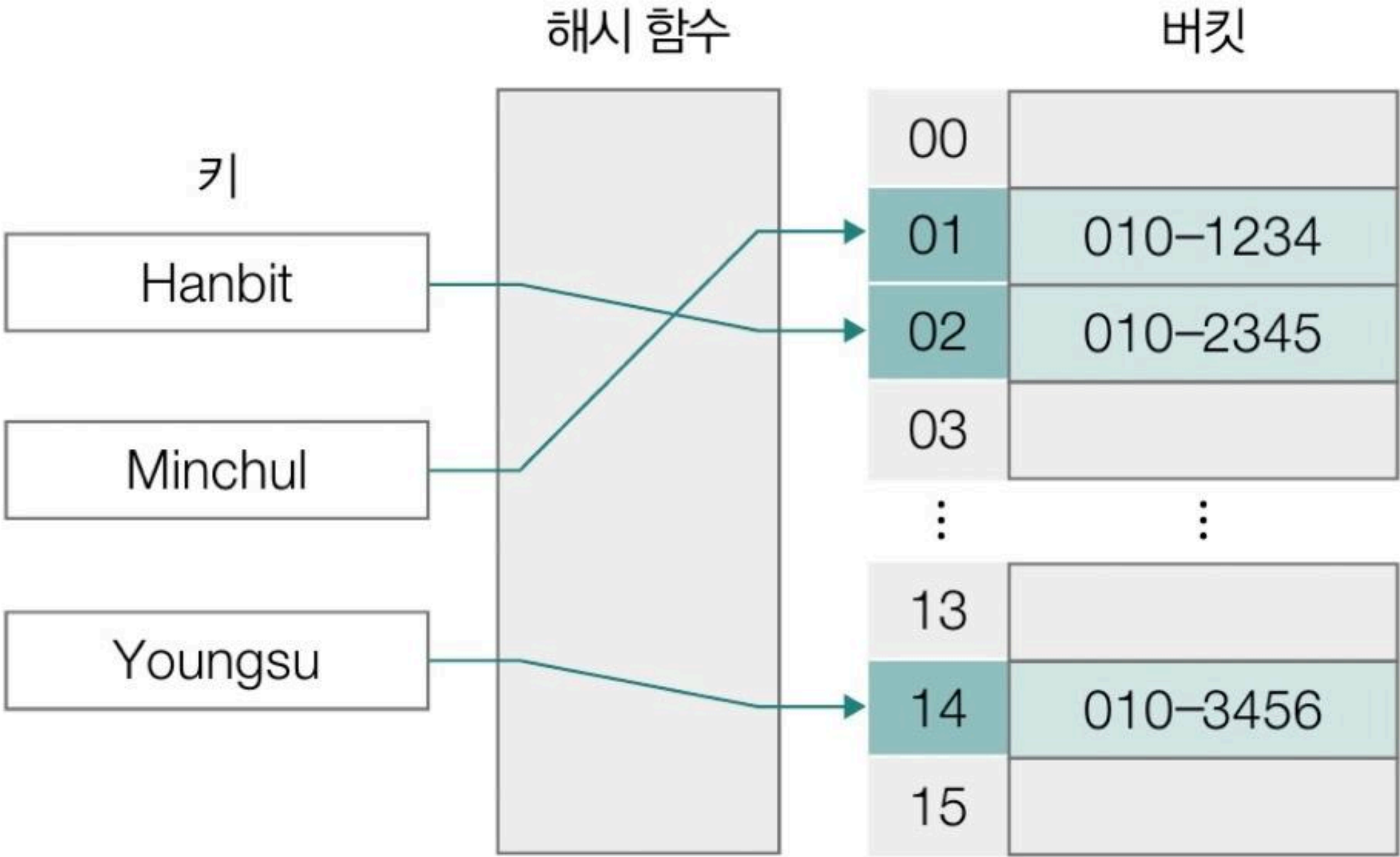
2. 해시 테이블의 구조와 동작



동작원리

- 해시 함수는 키를 인자로 활용하여, **버킷 배열의 인덱스**를 반환
- 키를 해시 함수에 통과시켜 원하는 버킷에 바로 접근 ($O(1)$)

2. 해시 테이블의 구조와 동작



로드팩터

- 해시 테이블에 저장된 데이터 수 / 버킷의 수
- 해시 테이블이 현재 얼마나 가득 차있는지에 대한 지표

→ 로드 팩터가 클수록 해시 테이블의 성능 ⬇

(예시) 저장된 데이터 수: 7개, 버킷 수(배열 크기): 10개

계산: $7 / 10 = 0.7$

→ 의미: "이 해시 테이블은 현재 70% 가득 찼다!"

→ 리해싱

2. 해시 테이블의 구조와 동작

- 데이터가 많아지면 해시 충돌 발생. 이 해시 충돌이 늘어나면?
 - 한 버킷에 여러 값이 줄줄이 연결됨(체이닝)
 - 검색 시간 증가
 - 더 이상 시간복잡도가 $O(1)$ 이 아닌 $O(n)$ 에 가까워짐
 - 해시 테이블 성능 저하
- 결론: 그래서 로드 팩터가 높으면 해시 테이블이 느려지는 것

3. 해시 함수

키(key)를 입력받아 버킷 배열의 인덱스(정수)로 변환해주는 함수

입력: 키 (문자열, 숫자 등 다양한 타입)



해시 함수



출력: 인덱스 (0, 1, 2, 3, ... table_size-1)

비유해본다면 ⬇️

키 = 손님 이름

해시 함수 = 좌석 배정 규칙

인덱스 = 좌석 번호

3. 해시 함수

해시 함수는 **목적**에 따라 크게 두 분류로 나뉜다.

① 자료구조용 간단한 해시 함수

② 암호학적 해시 함수

3. 해시 함수

① 자료구조용 간단한 해시 함수

목적: 데이터를 빠르게 찾기 위해 배열 인덱스로 변환

특징:

- ⚡ 속도가 최우선 (매우 간단한 연산)
- 🎲 균등 분산이 중요 (모든 인덱스에 골고루 분배)
- 🔄 충돌 허용 (같은 인덱스 나와도 OK → 체이닝 등으로 해결)
- 🔓 역추적 가능해도 무방 (보안 목적 아님)

3. 해시 함수

① 자료구조용 간단한 해시 함수

대표적인 방법: 나눗셈 방법(모듈러 연산), 곱셈 방법

$$h(\text{key}) = \text{key} \% \text{table_size}$$

(예시) • $\text{table_size} = 10$

• $\text{key} = 12345$

• $h(12345) = 12345 \% 10 = 5$



인덱스 5번에 저장

3. 해시 함수

② 암호학적 해시 함수

목적: 보안, 데이터 무결성 검증

특징:

- 🔒 단방향성 (해시값 → 원본 복원 불가능)
- 💥 눈사태 효과 (입력 1비트 변경 → 출력 완전히 변경)
- 🛡️ 충돌 저항성 (같은 해시값 만들기 극도로 어려움)
- 📏 고정 길이 출력 (SHA-256은 항상 256비트)
- 🐌 상대적으로 느림 (복잡한 연산)

3. 해시 함수

② 암호학적 해시 함수

대표적인 알고리즘: MD5, SHA-1, SHA-256(현재 널리 사용), SHA-512 등

예시) SHA-256

입력: "password"

출력: "5e884898da28047151d0e56f8dc6292773603d0d6aabbdd62a11ef721d1542d8"

입력: "passwor" (d 하나 빠짐)

출력: "3c8b3b1e91e3d5f2c9a0e4c8b7f5d3e1a2b3c4d5e6f7a8b9c0d1e2f3a4b5c6d7"

3. 해시 함수

② 암호학적 해시 함수

어디에 사용되는가?

- 비밀번호 저장 (원본 비밀번호 대신 해시값 저장)
- 파일 무결성 검증 (파일이 변조되었는지 확인)
- 블록체인, 디지털 서명 등

3. 해시 함수

좋은 해시 함수의 조건

① **균등 분산** - 모든 키가 각 인덱스에 균등하게 분배되어야 함

적절하지 않은 해시 함수 🙄

// 모든 키가 인덱스 0으로만 감

$h(key) = 0$

// 짝수만 사용 (0, 2, 4, 6, 8만 사용)

$h(key) = (key * 2) \% 10$

적절한 해시 함수 👍

// 모든 인덱스를 골고루 사용

$h(key) = key \% \text{prime_number}$

3. 해시 함수

좋은 해시 함수의 조건

② 빠른 계산 속도

: 복잡한 연산 없이 한 번의 간단한 연산으로 계산

👍 좋음: $\text{key} \% 10$ (나눗셈 한 번)

👎 나쁨: 복잡한 수학 함수 여러 개 사용

③ 결정적 (Deterministic)

: 같은 키는 항상 같은 인덱스 반환

$h(\text{"Alice"}) = 3$

$h(\text{"Alice"}) = 3$ // 항상 같아야 함!

해시 테이블을 사용하는 이유

빠른 검색 속도

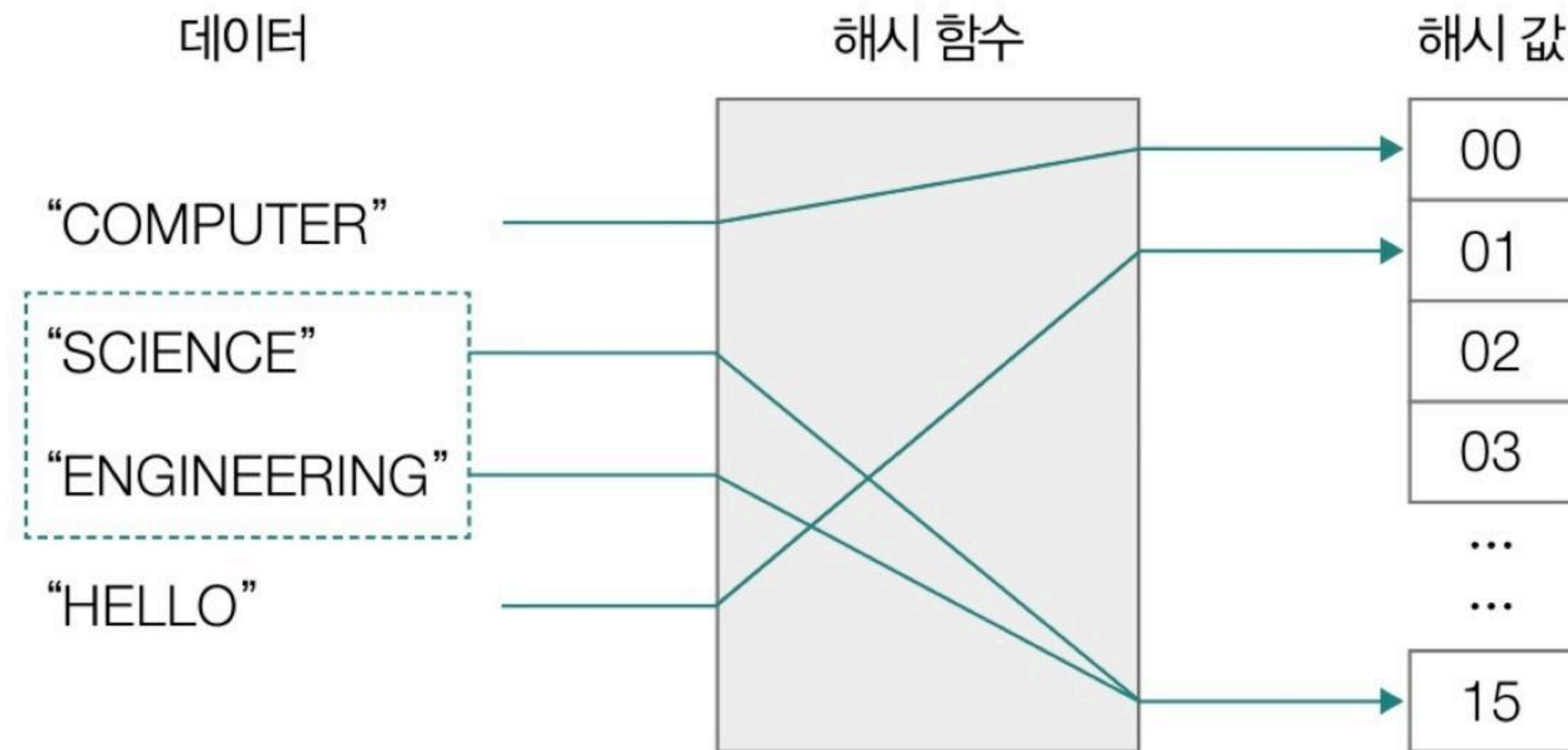
일반적인 상황에서 해시 테이블을 활용한 검색, 삽입, 삭제 연산의
시간 복잡도는 입력과 무관하게 $O(1)$

그렇다면 단점은?

속도가 빠른 만큼 상대적으로 많은 메모리 공간 소모, 해시 충돌 문제 해결 필요

4. 해시 충돌

서로 다른 키에 대해 같은 해시 값이 대응되는 상황



4. 해시 충돌

서로 다른 키에 대해 같은 해시 값이 대응되는 상황

< 실제 사례 >
구글과 CWI 암스테르담 연구소
: SHA-1 해시 함수가 사용하는
알고리즘에 해시 충돌이 발생할 수
있음을 입증한 바 있음

파일명	해시 알고리즘	해시 값
shattered-1.pdf 	MD5	ee4aa52b139d925f8d8884402b0a750c
	SHA-1	38762cf7f55934b34d179ae6a4c80cadccbb7f0a
	SHA-256	2bb787a73e37352f92383abe7e2902936d1059ad9f1ba6daaa9c1e58ee6970d0
shattered-2.pdf 	MD5	5bd9d8cabc46041579a311230539b8d1
	SHA-1	38762cf7f55934b34d179ae6a4c80cadccbb7f0a
	SHA-256	d4488775d29bdef7993367d541064dbdda50d383f89f0aa13a6ff2e0894ba5ff

4. 해시 충돌

해시 충돌의 원인?

- 해시 함수는 무한한 키를 유한한 개수의 인덱스로 매핑하므로, 충돌은 필연적으로 발생
- 키 값이 특정 패턴을 가지며, 이 패턴을 해시 함수가 충분히 분산시키지 못할 경우 일부 버킷에 키가 집중되어 충돌이 증가할 수 있음

예시) 짝수 키만 들어오는 경우

해시함수

$h(key) = key \% 8$

+

입력받는 키

2, 4, 6, 8, 10, 12

해시 결과

$$2 \% 8 = 2$$

$$4 \% 8 = 4$$

$$6 \% 8 = 6$$

$$8 \% 8 = 0$$

$$10 \% 8 = 2$$

$$12 \% 8 = 4$$

사용된 인덱스는 0, 2, 4, 6 뿐

즉, 전체 8칸 중 절반만 사용

버킷 활용률 50% 감소

4. 해시 충돌

해시 충돌의 원인?

- 해시 테이블의 크기를 잘못 선택하면(예: 적절한 prime(소수)가 아닌 경우) 특정 패턴의 키가 모여 충돌이 증가할 가능성이 커진다.

예시) 흔한 해시 함수 형태

$$h(\text{key}) = \text{key} \% \text{tableSize}$$

이때 tableSize가 합성수면,
키의 특정 공약수 패턴이 그대로 해시 결과에 반영된다.

tableSize = 10이라고 했을 때,
→ $h(\text{key}) = \text{key} \% 10$

입력받는 키

10, 20, 30, 40, 50, 60

해시 결과

10 % 10 = 0
20 % 10 = 0
30 % 10 = 0
40 % 10 = 0
...



👉 모든 키가 버킷 0으로 몰림

4. 해시 충돌

해시 충돌의 원인?

- 해시 테이블의 크기를 잘못 선택하면(예: 적절한 prime(소수)가 아닌 경우) 특정 패턴의 키가 모여 충돌이 증가할 가능성이 커진다.

예시) 흔한 해시 함수 형태

$$h(\text{key}) = \text{key} \% \text{tableSize}$$

tableSize = 11(소수)로 설정 !!

$$\rightarrow h(\text{key}) = \text{key} \% 11$$

입력받는 키

10, 20, 30, 40, 50, 60

해시 결과

10 % 11 = 10
20 % 11 = 9
30 % 11 = 8
40 % 11 = 7
50 % 11 = 6

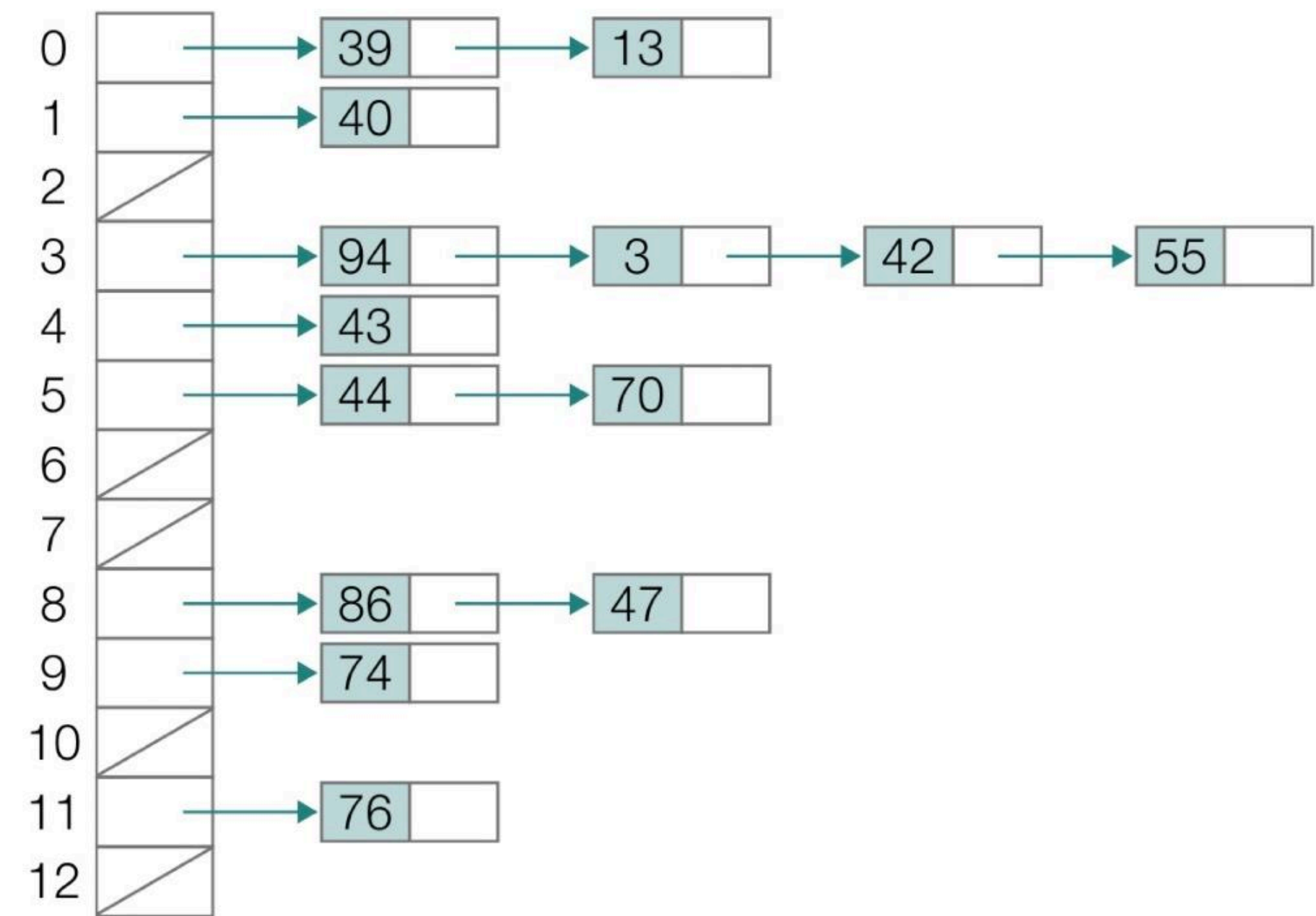


👉 버킷이 바르게 분산

5. 해시 충돌 해결 방법

(1) 체이닝

- 충돌이 발생한 데이터를 연결 리스트로 추가하는 방법
- 하나의 테이블 인덱스에 여러 데이터가 연결 리스트의 노드로서 존재
- 충돌이 발생할 때마다 연결 리스트의 노드를 추가하는 건 빠른 속도 보장 X
→ 해시 테이블의 장점 살리지 못함



5. 해시 충돌 해결 방법

(2) 개방 주소법

- 충돌이 발생한 버킷의 인덱스가 아닌 다른 인덱스에 데이터를 저장하는 방법
- 비어 있는 다른 버킷의 인덱스를 찾는 과정 = '조사'한다
- 조사 방법에 따라 나뉘는데, 그 중 하나가 선형 조사법

5. 해시 충돌 해결 방법

(2) 개방 주소법 - 선형 조사법

- 충돌이 발생한 인덱스의 다음 인덱스부터 순차적으로 가용한 인덱스를 찾아 나서는 방법
- 단점: 군집화 - 충돌이 발생하는 인덱스 인근에 충돌이 발생한 여러 데이터가 몰려 저장될 수 있다.

0	010-1234
1	
2	010-1234
3	010-1111
4	
5	
6	010-3456
7	010-4567
8	
9	

충돌

010-1111 →

0	010-1234
1	
2	010-1234
3	010-1111
4	
5	
6	010-3456
7	010-4567
8	010-2222
9	

충돌

010-2222 →

5. 해시 충돌 해결 방법

(3) 이중 해싱

- 2개의 해시 함수를 사용
- 충돌이 발생했을 때 다른 해시 함수(보조 해시 함수)에 대한 해시 값만큼 떨어진 거리에 위치한 인덱스를 찾는 방법

ex. 충돌 발생 → $f(\text{key}) + g(\text{key})$ 에서 인덱스를 찾음

→ 여기서도 충돌 발생 시 $f(\text{key}) + 2g(\text{key}), f(\text{key}) + 3g(\text{key}), \dots$

마무리

해시 테이블의 빠른 검색 속도로 인한 뛰어난 성능

but,

좋은 해시 함수, 적절한 테이블 크기, 충돌 해결 전략이 꼭 필요

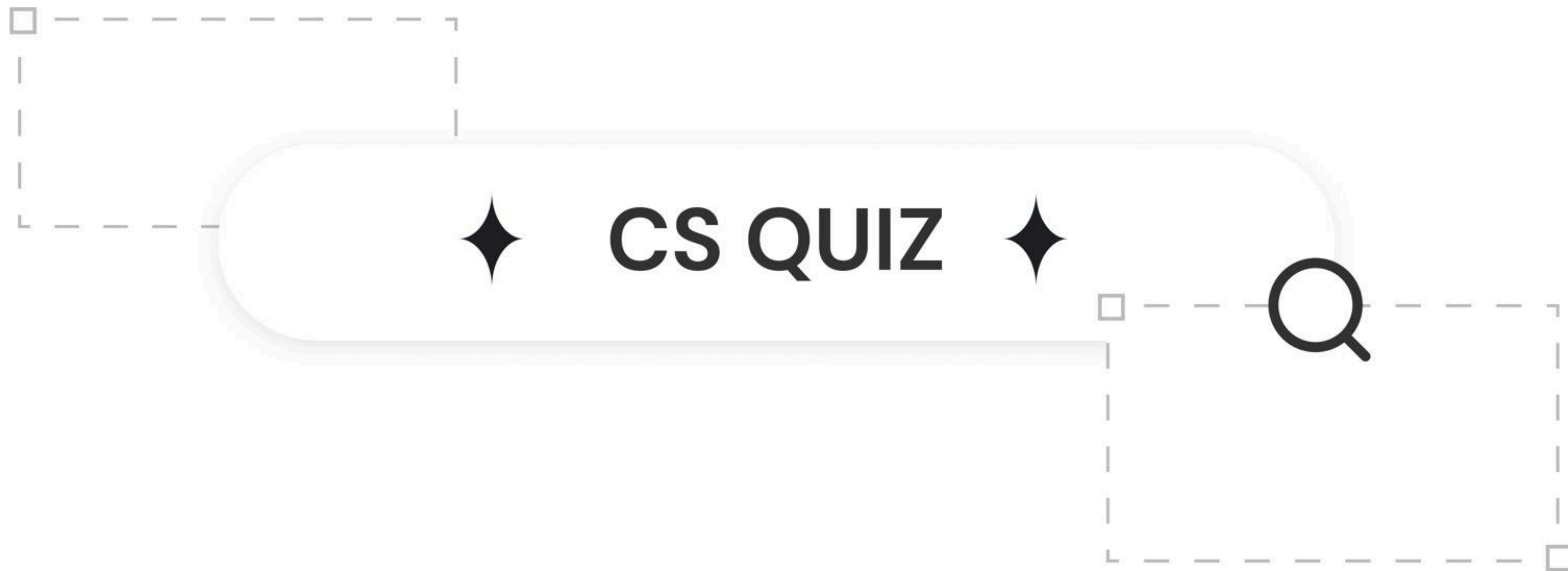


코테이토의 새로운 퀴즈 사이트 MAIT



<https://mait.kr/>





Q.1

1. 해시 테이블을 사용하는 이유로 가장 적절한 것은?

- ① 데이터를 정렬된 상태로 유지하기 위해
- ② 검색, 삽입, 삭제 연산을 빠르게 수행하기 위해
- ③ 메모리 사용량을 최소화하기 위해
- ④ 항상 충돌을 발생시키지 않기 위해



A.1

1. 해시 테이블을 사용하는 이유로 가장 적절한 것은?

- ① 데이터를 정렬된 상태로 유지하기 위해
- ② 검색, 삽입, 삭제 연산을 빠르게 수행하기 위해
- ③ 메모리 사용량을 최소화하기 위해
- ④ 항상 충돌을 발생시키지 않기 위해

해시 테이블의 가장 큰 장점은 평균 시간 복잡도 $O(1)$ 의 빠른 연산 속도이다.
정렬 유지(BST 등)나 메모리 절약이 주목적은 아니다.

Q.2

2. 다음 설명이 올바른면 O, 그렇지 않으면 X를 고르시오.

해시 테이블은 항상 검색 시간이 $O(1)$ 이다.



A.2

2. 다음 설명이 올바른면 O, 그렇지 않으면 X를 고르시오.

해시 테이블은 항상 검색 시간이 $O(1)$ 이다.

정답: ✖

평균은 $O(1)$ 이지만, 충돌이 많아지면 최악의 경우 $O(n)$ 이 될 수 있다.

Q.3

3. 해시 테이블의 '로드 팩터'에 대한 설명으로 옳은 것은?

- ① 해시 충돌이 발생했을 때 연결된 리스트의 길이를 의미한다.
- ② 항상 1보다 커야 해시 테이블의 성능이 보장된다.
- ③ 저장된 데이터 수를 버킷 수로 나눈 값으로 테이블의 꽉 찬 정도를 의미한다.
- ④ 해시 함수의 연산 속도를 나타내는 수치이다.



A.3

3. 해시 테이블의 '로드 팩터'에 대한 설명으로 옳은 것은?

- ① 해시 충돌이 발생했을 때 연결된 리스트의 길이를 의미한다.
- ② 항상 1보다 커야 해시 테이블의 성능이 보장된다.
- ③ 저장된 데이터 수를 버킷 수로 나눈 값으로 테이블의 꽉 찬 정도를 의미한다.
- ④ 해시 함수의 연산 속도를 나타내는 수치이다.

로드 팩터는 저장된 데이터 수 / 버킷 수로 계산되며, 해시 테이블이 얼마나 가득 찼는지를 나타낸다.

데이터 수와 버킷 수의 비율을 통해 충돌 가능성과 저하 시점을 예측할 수 있다.

Q.4

4. 해시 테이블에 데이터를 삽입하는 과정을 순서대로 나열한 것으로 옳은 것은?

- A. 해시 함수 계산
- B. 데이터 저장
- C. 충돌 확인
- D. 키(key) 입력

- ① $D \rightarrow A \rightarrow B \rightarrow C$
- ② $D \rightarrow A \rightarrow C \rightarrow B$
- ③ $A \rightarrow D \rightarrow C \rightarrow B$
- ④ $C \rightarrow D \rightarrow A \rightarrow B$
- ⑤ $D \rightarrow C \rightarrow A \rightarrow B$



A.4

4. 해시 테이블에 데이터를 삽입하는 과정을 순서대로 나열한 것으로 옳은 것은?

- A. 해시 함수 계산
- B. 데이터 저장
- C. 충돌 확인
- D. 키(key) 입력

- ① $D \rightarrow A \rightarrow B \rightarrow C$
- ② $D \rightarrow A \rightarrow C \rightarrow B$
- ③ $A \rightarrow D \rightarrow C \rightarrow B$
- ④ $C \rightarrow D \rightarrow A \rightarrow B$
- ⑤ $D \rightarrow C \rightarrow A \rightarrow B$

키(key) 입력 → 해시 함수 계산 → 충돌 확인 → 데이터 저장

Q.5

5. 다음 중 자료구조용 해시 함수에서 중요하게 고려하지 않아도 되는 것은?

- ① 같은 키에 대해 항상 같은 해시 값을 반환해야 한다.
- ② 해시 충돌이 발생하더라도 이를 허용하고 해결 전략을 사용한다.
- ③ 계산 속도가 빠르고 연산이 단순해야 한다.
- ④ 입력 값이 아주 조금만 바뀌어도 출력 값이 완전히 달라져야 한다.



A.5

5. 다음 중 자료구조용 해시 함수에서 중요하게 고려하지 않아도 되는 것은?

- ① 같은 키에 대해 항상 같은 해시 값을 반환해야 한다.
- ② 해시 충돌이 발생하더라도 이를 허용하고 해결 전략을 사용한다.
- ③ 계산 속도가 빠르고 연산이 단순해야 한다.
- ④ 입력 값이 아주 조금만 바뀌어도 출력 값이 완전히 달라져야 한다.

출력 값의 완전한 변화(눈사태 효과)는 보안 목적의 암호학적 해시 함수에서 중요한 성질이며, 자료구조용 해시 함수에서는 속도와 균등 분산이 더 중요하다.

Q.6

6. 해시 충돌이 발생했을 때, 충돌이 난 인덱스에 연결 리스트를 사용해 데이터를 저장하는 방식을 무엇이라고 하는가? (주관식)



A.6

6. 해시 충돌이 발생했을 때, 충돌이 난 인덱스에 연결 리스트를 사용해 데이터를 저장하는 방식을 무엇이라고 하는가? (주관식)

체이닝

체이닝은 하나의 버킷에 여러 데이터를 연결 리스트 형태로 저장하여 충돌을 해결하는 방식이다.

Q.7

(MAIT용 문제)

7. 해시 테이블에서 키(Key)의 역할로 가장 적절한 것은?

- ① 데이터를 정렬하기 위한 기준
- ② 해시 충돌을 방지하기 위한 값
- ③ 해시 함수의 입력으로 사용되는 값
- ④ 버킷의 개수를 결정하는 값



A.7

(MAIT용 문제)

7. 해시 테이블에서 키(Key)의 역할로 가장 적절한 것은?

- ① 데이터를 정렬하기 위한 기준
- ② 해시 충돌을 방지하기 위한 값
- ③ 해시 함수의 입력으로 사용되는 값
- ④ 버킷의 개수를 결정하는 값

키는 해시 함수의 입력값이며, 이를 통해 버킷 배열의 인덱스를 계산한다.

Q.8

(MAIT용 문제)

8. 해시 테이블의 크기를 정할 때 소수를 사용하는 주된 이유는?

- ① 특정 패턴의 공약수를 끊어주어 데이터를 고르게 분산시키기 위해서이다.
- ② 해시 함수의 연산 속도를 물리적으로 빠르게 하기 때문이다.
- ③ 메모리 공간을 최소한으로 사용하기 때문이다.
- ④ 데이터의 보안성을 높여 암호화에 유리하기 때문이다.



A.8

(MAIT용 문제)

8. 해시 테이블의 크기를 정할 때 소수를 사용하는 주된 이유는?

- ① 특정 패턴의 공약수를 끊어주어 데이터를 고르게 분산시키기 위해서이다.
- ② 해시 함수의 연산 속도를 물리적으로 빠르게 하기 때문이다.
- ③ 메모리 공간을 최소한으로 사용하기 때문이다.
- ④ 데이터의 보안성을 높여 암호화에 유리하기 때문이다.

해시 함수의 핵심은 데이터를 겹치지 않게 잘 뿌리는 것, 소수는 데이터가 가진 규칙성을 파괴해서 골고루 섞어주는 역할을 한다.

Q.9

9. 해시 충돌 해결 방법 중 이중 해싱에 대한 설명으로 가장 적절한 것은?

- ① 충돌이 발생하면 바로 옆자리의 빈 버킷을 찾아 저장한다.
- ② 충돌이 발생하면 해당 버킷에 연결 리스트를 만들어 데이터를 추가한다.
- ③ 해시 함수를 두 개 사용하여, 충돌 시 두 번째 해시 함수 값만큼 떨어진 위치를 찾는다.
- ④ 테이블의 크기를 두 배로 늘려 충돌을 방지한다.



A.9

9. 해시 충돌 해결 방법 중 이중 해싱에 대한 설명으로 가장 적절한 것은?

- ① 충돌이 발생하면 바로 옆자리의 빈 버킷을 찾아 저장한다.
- ② 충돌이 발생하면 해당 버킷에 연결 리스트를 만들어 데이터를 추가한다.
- ③ 해시 함수를 두 개 사용하여, 충돌 시 두 번째 해시 함수 값만큼 떨어진 위치를 찾는다.
- ④ 테이블의 크기를 두 배로 늘려 충돌을 방지한다.

이중 해싱이란 해시 함수를 두 개 사용해서, 충돌이 발생했을 때 다른 해시 함수(보조 해시 함수)에 대한 해시 값만큼 떨어진 거리에 위치한 인덱스를 찾는 방법.

🗨️ 14 🍷 356 1번은 선형 조사법, 2번은 체이닝

Q.10

10. 개방 주소법 중 선형 조사법을 사용할 때 발생할 수 있는 문제점으로, 충돌이 발생한 인덱스 인근에 데이터가 몰려 성능이 악화되는 현상은?

- ① 데이터의 군집화 (Clustering)
- ② 메모리 누수 (Memory Leak)
- ③ 스택 오버플로우 (Stack Overflow)
- ④ 무한 루프 (Infinite Loop)



A.10

10. 개방 주소법 중 선형 조사법을 사용할 때 발생할 수 있는 문제점으로, 충돌이 발생한 인덱스 인근에 데이터가 몰려 성능이 악화되는 현상은?

- ① 데이터의 군집화 (Clustering)
- ② 메모리 누수 (Memory Leak)
- ③ 스택 오버플로우 (Stack Overflow)
- ④ 무한 루프 (Infinite Loop)

Q.11

11. 다음과 같은 상황에서 발생할 수 있는 문제로 가장 적절한 것은?

해시 함수: $h(\text{key}) = \text{key} \% 8$
입력되는 키: 2, 4, 6, 8, 10, 12

- ① 모든 버킷이 균등하게 사용된다.
- ② 해시 함수 계산 비용이 너무 크다.
- ③ 암호학적 안정성이 떨어진다.
- ④ 특정 버킷에 키가 집중되어 충돌이 증가한다.



A.11

11. 다음과 같은 상황에서 발생할 수 있는 문제로 가장 적절한 것은?

해시 함수: $h(\text{key}) = \text{key} \% 8$

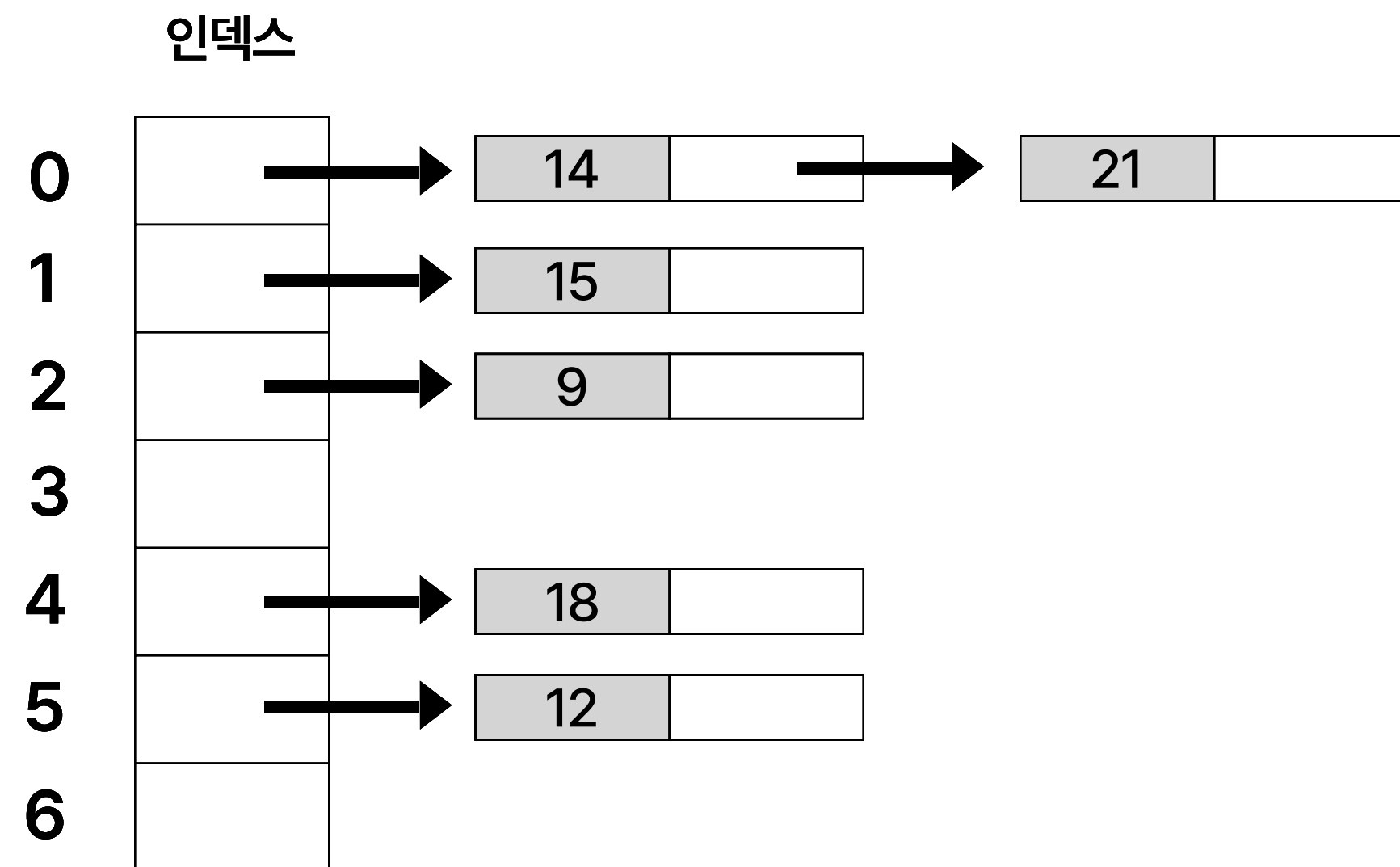
입력되는 키: 2, 4, 6, 8, 10, 12

- ① 모든 버킷이 균등하게 사용된다
- ② 해시 함수 계산 비용이 너무 크다
- ③ 암호학적 안정성이 떨어진다
- ④ 특정 버킷에 키가 집중되어 충돌이 증가한다

0, 2, 4, 6 인덱스만 사용되고 나머지는 비게 된다.
이는 해시 충돌 증가 + 버킷 활용률 감소로 이어진다.

Q.12

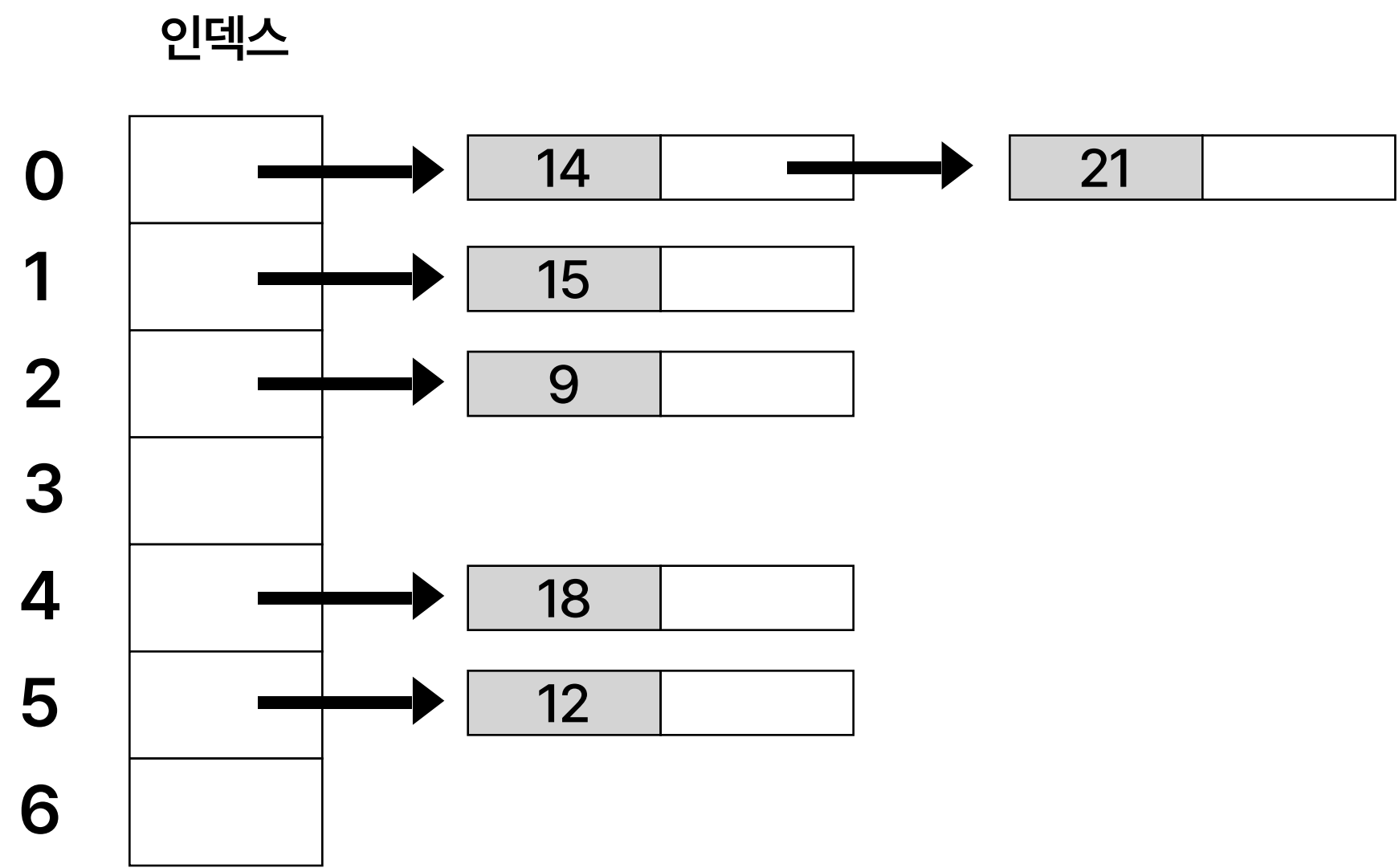
12. 다음은 크기가 7인 해시테이블에 체이닝 방식으로 데이터를 저장한 상태이며, 해시 함수는 $h(\text{key}) = \text{key} \% 7$ 을 사용한다.
이때 키 값이 21인 데이터를 찾기 위해 탐색하는 과정을 가장 올바르게 설명한 것은?



- ① 해시 함수 계산 → 인덱스 0 접근 → 21 즉시 반환
- ② 해시 함수 계산 → 인덱스 7 접근 → 21 반환
- ③ $21 \% 7 = 0$ 계산 → 인덱스 0의 모든 노드 순회 → 21 발견 후 반환
- ④ 선형 탐색으로 인덱스 0부터 6까지 순회 후 21 발견 후 반환

A.12

12. 다음은 크기가 7인 해시테이블에 체이닝 방식으로 데이터를 저장한 상태이며, 해시 함수는 $h(key) = key \% 7$ 을 사용한다.
이때 키 값이 21인 데이터를 찾기 위해 탐색하는 과정을 가장 올바르게 설명한 것은?



- ① 해시 함수 계산 → 인덱스 0 접근 → 21 즉시 반환
- ② 해시 함수 계산 → 인덱스 7 접근 → 21 반환
- ③ $21 \% 7 = 0$ 계산 → 인덱스 0의 모든 노드 순회 → 21 발견 후 반환
- ④ 선형 탐색으로 인덱스 0부터 6까지 순회 후 21 발견 후 반환